

[CS230] Structuring Machine Learning Projects: Error Analysis

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-6. ML Strategy Basics - *Completed*

Chapter 7. Error Analysis (Current Unit)

- 7.1 Manual Error Analysis
 - * Philosophy: Don't Guess, Look
 - * Ceiling Analysis (Prioritization)
 - * Incorrectly Labeled Data Strategy
 - * Implementation: Analysis Tool
- 7.2 Training vs Dev/Test Distribution Mismatch - *Upcoming*
- 7.3 Transfer Learning & Multi-task Learning - *Upcoming*

지난 시간 복습 및 연결

우리는 최적의 모델을 선택하는 방법을 배웠습니다. 하지만 선택된 모델도 목표 성능(예: 99%)에는 도달하지 못했을 수 있습니다. 이때 많은 엔지니어들이 "직감"에 의존합니다. "개가 고양이처럼 보이네? 개 데이터를 더 모으자", "흐려서 그런가? 포토샵 전처리를 하자." 하지만 이것이 수개월을 낭비하는 최악의 결정이 될 수 있습니다. 우리는 직감이 아니라 '데이터'가 말하게 해야 합니다. 오늘 배울 에러 분석은 가장 효율적인 성능 향상 경로를 알려주는 나침반입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 틀린 데이터를 직접 분석하여 프로젝트의 우선순위를 정하는 전략을 다룹니다.

- **철학:** "추측하지 말고 확인하라(Don't guess, look)." 오분류된 데이터를 직접 눈으로 확인합니다.
- **천장 분석:** 특정 문제를 해결했을 때 성능이 얼마나 오를지 상한선(Ceiling)을 계산합니다.
- **라벨 오류:** 정답 라벨 자체가 틀린 경우(Incorrect Label), 이를 수정해야 할지 판단하는 기준을 배웁니다.
- **구현:** 에러 리포트를 자동으로 생성하는 Python 클래스를 작성합니다.

2 Essential Terminology

용어	의미	활용
Ceiling Analysis	성능 향상의 최대치 분석	"이거 고치면 몇 점 오르지?" 계산
Misclassified	오분류된 데이터	모델이 틀린 것만 모아놓은 집합
Incorrect Label	잘못된 정답지	사람이 실수로 라벨링을 잘못된 경우

3 Core Concepts: 데이터가 말하게 하라

3.1 1. The Philosophy: Don't Guess, Look

모델 정확도가 90%입니다. (에러율 10%). 팀원이 제안합니다. "흐릿한(Blurry) 사진 때문에 틀리는 것 같아요. 흐림 제거 모델을 만듭시다! (예상 소요: 3개월)" 이 제안을 수락해야 할까요? 천장 분석(Ceiling Analysis)을 해보기 전엔 모릅니다.

3.2 2. Ceiling Analysis (천장 분석)

Dev Set에서 모델이 틀린 샘플 100개를 무작위로 뽑아 엑셀에 정리합니다.

□ 분석 결과 시뮬레이션 (전략 가이드)

에러 원인 (Category)	비율 (Count)	Ceiling (예상 향상)
흐릿함 (Blurry)	5%	$10\% \times 0.05 = 0.5\%$
배경 노이즈 (Noise)	60%	$10\% \times 0.60 = 6.0\%$
고양이 닮은 개	35%	$10\% \times 0.35 = 3.5\%$

결론: 흐림 제거 모델을 완벽하게 만들어도 성능은 고작 **0.5%** 오릅니다. 3개월을 낭비할 뻔했습니다. 우리는 '배경 노이즈' 문제(6.0% 향상 가능)에 집중해야 합니다.

3.3 3. Incorrectly Labeled Data (라벨 오류)

데이터를 보다 보니, 모델은 맞았는데 사람이 정답을 잘못 달아놓은 경우를 발견했습니다. 고쳐야 할까요?

- **Training Set:** 딥러닝은 무작위 오류(Random Error)에 강합니다. 데이터가 많다면 무시해도 됩니다. (단, 체계적 오류는 수정 필수)
- **Dev/Test Set:** 에러 분석 표에 'Incorrect Label' 열을 추가합니다.
 - 만약 이 비율이 전체 에러의 상당수라면(예: 에러의 30%), 반드시 고쳐야 합니다.
 - 주의: Dev와 Test는 항상 동시에 고쳐야 분포가 유지됩니다.

4 Implementation: Error Report Generator

에러 분석을 도와주는 자동화 도구입니다.

```
1 import pandas as pd
2 import numpy as np
3
4 class ErrorAnalyzer:
```

```

5  def __init__(self, y_true, y_pred):
6      self.y_true = np.array(y_true)
7      self.y_pred = np.array(y_pred)
8      # 틀린 인덱스 추출
9      self.error_indices = np.where(self.y_true != self.y_pred)[0]
10     self.total_errors = len(self.error_indices)
11     self.total_samples = len(y_true)
12
13     def generate_report(self, manual_tags):
14         """
15         manual_tags: {index: ['Blurry', 'Noise'], ...}
16         """
17         counts = {}
18         for tags in manual_tags.values():
19             for tag in tags:
20                 counts[tag] = counts.get(tag, 0) + 1
21
22         df = pd.DataFrame(list(counts.items()), columns=['Category', 'Count'])
23
24         # Ceiling Analysis 계산
25         # 전체 데이터 대비 성능 향상 가능치
26         df['Ceiling (%)'] = (df['Count'] / self.total_samples) * 100
27
28         # 정렬
29         df = df.sort_values(by='Count', ascending=False)
30         return df
31
32 # --- 실행 ---
33 if __name__ == "__main__":
34     # 전체 데이터 개1000 중 에러 개라고100 가정
35     y_true = np.zeros(1000)
36     y_pred = np.zeros(1000)
37     y_pred[:100] = 1 # 개100 틀림
38
39     analyzer = ErrorAnalyzer(y_true, y_pred)
40
41     # 사람이 직접 보고 태깅했다고 가정 엑셀(연동)
42     tags = {
43         0: ['Blurry'], 1: ['Noise'], 2: ['Blurry', 'Noise'],
44         # ... 생략() ...
45         99: ['Noise']
46     }
47     # 가정(: Blurry 개5, Noise 개60)
48
49     # 리포트 생성
50     # df = analyzer.generate_report(tags)
51     # print(df)

```

Listing 1: Error Analysis Tool

5 FAQ & Pitfalls

Dev Set만 고치면 안 되나요? (오해 방지 가이드)

절대 안 됩니다. Dev Set의 라벨만 고치고 Test Set을 그대로 두면, 두 데이터셋의 분포가 달라집니다(Distribution Mismatch). 평가의 신뢰도가 깨집니다. 귀찮더라도 Dev와 Test는 한 몸처럼 다뤄야 합니다.

Q. 몇 개나 분석해야 하나요?

A. 보통 100개 정도면 충분한 통계적 인사이트를 얻을 수 있습니다. 혼자서 100개 보는 데 1 2

시간이면 됩니다. 팀원들과 100개씩 나눠서 보면 더 좋습니다.

다음 단계 (Next Step)

에러 분석을 통해 '무엇'을 고쳐야 할지 알게 되었습니다. 그런데 분석 결과, "훈련 데이터와 검증 데이터의 분포가 너무 다르다"는 결론이 나오면 어떻게 해야 할까요? (예: 훈련은 고화질, 검증은 저화질)

이것은 단순한 과대 적합과는 다른 차원의 문제입니다. 다음 시간에는 [Training vs Dev/Test Distribution Mismatch] 문제를 진단하고 해결하는 고급 전략인 'Training-Dev Set'의 개념에 대해 배우겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Don't Guess:** 직감이 아닌 데이터를 보고 결정하라.
2. **Ceiling Analysis:** 특정 문제를 해결했을 때 얻을 수 있는 최대 이익(ROI)을 계산하라.
3. **Incorrect Label:** 전체 에러 중 비중이 높다면 수정하라. 단, Dev/Test를 동시에 수정해야 한다.
4. **Spreadsheet:** 엑셀 등을 활용해 팀원과 에러 원인을 공유하라.